Assignment 4

Name: Riddhi Agarwal
Roll Number: 2024CSB046

Problem 1

Question

An online educational analytics platform conducts national-level mock examinations for various competitive tests. In each examination session, approximately 110,000 students participate simultaneously. Once the test concludes, the scores obtained by all students are collected and stored on the server in the form of an unsorted integer array. To generate real-time performance analytics, such as identifying the central tendency of student performance, the platform needs to compute the median score as efficiently as possible. Since the dataset is large, the choice of algorithm has a significant impact on both execution time and memory utilization.

Write a C program to determine the median score using two different approaches, as described below:

i) Fixed Pivot-Based Approach: Employs a divide-and-conquer approach in which the pivot is selected in a fixed manner.

ii) Quickselect-Based Approach: Determine the median using the Quickselect algorithm, which finds the required order statistic without fully sorting the array.

After implementing both methods, perform a comparative analysis of the two approaches by evaluating and discussing their:

i) Time complexity in the best case, average case, and worst case

ii) Space complexity in the best case, average case, and worst case

Finally, justify which approach is more suitable for handling large-scale, real-time examination data and explain your conclusion with appropriate reasoning.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define STUDENT_COUNT 110000

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
```

```c
}

int partitionFixed(int *a, int low, int high)
{
    int pivot = a[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (a[j] <= pivot)
        {
            swap(&a[++i], &a[j]);
        }
    }
    swap(&a[++i], &a[high]);
    return i;
}

int partitionRandom(int *a, int low, int high)
{
    int r = low + rand() % (high - low + 1);
    swap(&a[r], &a[high]);
    return partitionFixed(a, low, high);
}

int select(int *a, int low, int high, int k, int useRandom)
{
    if (low <= high)
    {
        int pi = useRandom ? partitionRandom(a, low, high) :
   partitionFixed(a, low, high);

        if (pi == k)
            return a[pi];
        if (pi > k)
            return select(a, low, pi - 1, k, useRandom);
        return select(a, pi + 1, high, k, useRandom);
    }
    return -1;
}

int main()
{
    int n = STUDENT_COUNT;
    int *a = malloc(n * sizeof(int));
    int *b = malloc(n * sizeof(int));
    srand(time(NULL));

    for (int i = 0; i < n; i++)
    {
        a[i] = rand() % 101;
        b[i] = a[i];
```

```c
    }

    clock_t start = clock();
    int median1 = select(a, 0, n - 1, n / 2, 0); //0 for fixed partition and 1 for random partition
    clock_t end = clock();
    printf("Fixed Pivot Median: %d | Time: %f s\n", median1, (double)(end - start) / CLOCKS_PER_SEC);

    start = clock();
    int median2 = select(b, 0, n - 1, n / 2, 1);
    end = clock();
    printf("Randomized Quickselect Median: %d | Time: %f s\n", median2, (double)(end - start) / CLOCKS_PER_SEC);

    free(a);
    free(b);
    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment4> gcc Ques1.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment4> ./a.exe
Fixed Pivot Median: 50 | Time: 0.005000 s
Randomized Quickselect Median: 50 | Time: 0.012000 s
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment4>
```

Figure 1: Output for Problem 1.

Analysis

- **Time Complexity:** Both approaches have an **Average Case** complexity of $O(n)$. However, the Fixed Pivot approach has a **Worst Case** of $O(n^2)$ if the input is already sorted or nearly sorted. The Randomized Quickselect significantly reduces the probability of hitting the worst-case scenario.
- **Space Complexity:** Both are $O(1)$ auxiliary space if implemented iteratively, but $O(n)$ in the worst-case stack depth for recursion ($O(\log n)$ average).
- **Justification:** For large-scale real-time data, the **Randomized Quickselect** is more suitable as it provides a probabilistic guarantee against the $O(n^2)$ worst-case performance, ensuring consistent speed regardless of input distribution.

Problem 2

Question

A national digital census and survey platform periodically gathers extensive demographic and economic information from 120000 households distributed across the country. Among the various data points collected, household income values are stored on the server in the form of a large unsorted array. After the completion of each survey cycle, policymakers and analysts require reliable statistical indicators to support data-driven decision-making. One of the most critical measures is the median household income, as it provides a robust representation of central tendency and is not unduly influenced by extreme income values.

Given the enormous size of the dataset, sorting the entire array to compute the median becomes computationally expensive and impractical for real-time or near real-time analysis. To overcome this limitation, the analytics team opts for a deterministic linear-time selection algorithm, commonly referred to as the Median of Medians method, which guarantees $O(n)$ worst-case time complexity.

Write a C program that computes the median household income using the Median of Medians algorithm. The implementation should avoid full-array sorting and ensure deterministic performance irrespective of input distribution.

After successfully implementing the algorithm, perform a comparative performance analysis between the Median of Medians method and the Quickselect-based median finding method. The comparison must include a detailed evaluation of both algorithms in terms of their:

i) Best-case, average-case, and worst-case time complexities

ii) Best-case, average-case, and worst-case space complexities

Finally, represent the comparative results graphically by plotting the complexity values on a two-dimensional (2D) plane, and critically analyze the plots to justify the suitability of each method for large-scale census data processing.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define DATASET_SIZE 120000

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
```

```
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

int partition(int arr[], int l, int r, int x) {
    int i;
    for (i = l; i < r; i++)
        if (arr[i] == x)
            break;
    swap(&arr[i], &arr[r]);

    i = l;
    for (int j = l; j <= r - 1; j++) {
        if (arr[j] <= x) {
            swap(&arr[i], &arr[j]);
            i++;
        }
    }
    swap(&arr[i], &arr[r]);
    return i;
}

int medianOfMedians(int arr[], int l, int r, int k) {
    if (k > 0 && k <= r - l + 1) {
        int n = r - l + 1;

        int i, median[(n + 4) / 5];

        for (i = 0; i < n / 5; i++) {
            insertionSort(arr + l + i * 5, 5);
            median[i] = arr[l + i * 5 + 2];
        }

        if (i * 5 < n) {
            insertionSort(arr + l + i * 5, n % 5);
            median[i] = arr[l + i * 5 + (n % 5) / 2];
            i++;
        }

        int medOfMed = (i == 1) ? median[i - 1] :
                                  medianOfMedians(median, 0, i - 1, i
    / 2);

        int pos = partition(arr, l, r, medOfMed);

        if (pos - l == k - 1)
            return arr[pos];
```

```c
        if (pos - l > k - 1)
            return medianOfMedians(arr, l, pos - 1, k);

        return medianOfMedians(arr, pos + 1, r, k - pos + l - 1);
    }

    return -1;
}

int main() {
    int *household_incomes = (int *)malloc(DATASET_SIZE * sizeof(int));
    if (household_incomes == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    srand(time(0));

    for (int i = 0; i < DATASET_SIZE; i++) {
        household_incomes[i] = 10000 + rand() % 190001;
    }

    int median = medianOfMedians(household_incomes, 0, DATASET_SIZE - 1, (DATASET_SIZE + 1) / 2);

    printf("Median Household Income: $%d\n", median);

    free(household_incomes);
    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment4> gcc .\Ques2.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment4> ./a.exe
Median Household Income: $26413
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment4>
```

Figure 2: Output for Problem 2.

Complexity Visualization

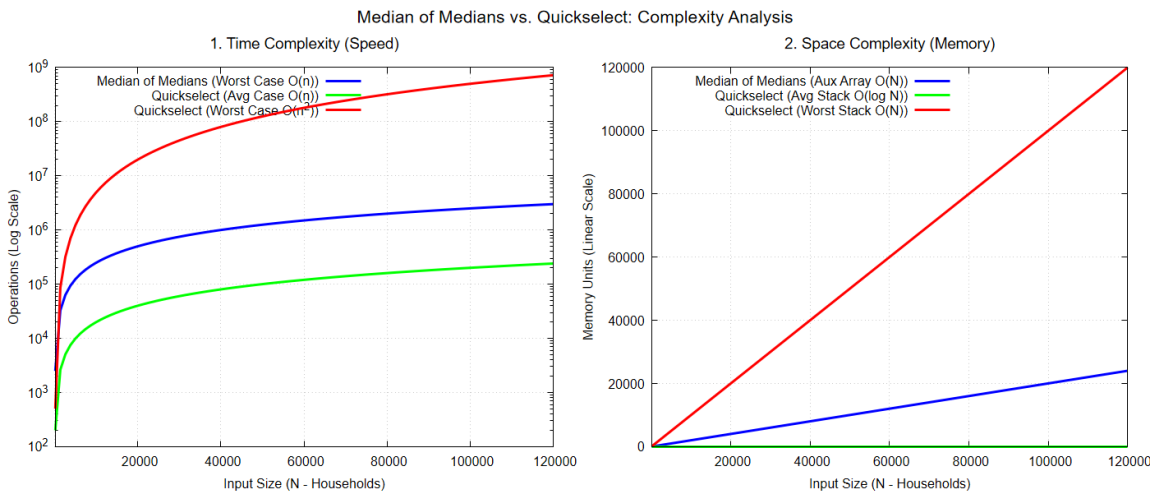

Figure 3: Complexity comparison: Median of Medians vs. Quickselect.

Analysis

- **Deterministic Performance:** Unlike Quickselect, Median of Medians guarantees $O(n)$ **worst-case** time complexity by choosing a pivot that is guaranteed to be around the 30th to 70th percentile.
- **Space Complexity:** Requires $O(n)$ auxiliary space for storing medians of groups at each level of recursion.
- **Suitability:** For census data, where the distribution might be unknown or adversarial (highly skewed), this method provides the highest reliability. However, due to higher constant factors, it may be slower than randomized Quickselect on average.

Problem 3

Question

A financial technology analytics firm processes transaction data from two independent banking systems. Each bank stores the daily transaction amounts of its customers in a sorted array (sorted in non-decreasing order). At the end of each day, the firm must compute the median transaction value across both banks to generate risk and liquidity assessment reports. Due to strict performance requirements and memory constraints, the firm cannot afford to merge the two arrays into a single array. Instead, the median must be computed directly from the two sorted datasets using an efficient algorithm.

Write a C program to compute the median of the combined dataset formed by two given sorted arrays, without merging the arrays into a single structure. Further, examine whether it is possible to design an algorithm that solves this problem with logarithmic time complexity. If such an algorithm exists, implement your solution strictly following that approach and justify its efficiency through appropriate time complexity analysis.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define MIN(a, b) ((a) < (b) ? (a) : (b))

double findMedianSortedArrays(int *arr1, int n, int *arr2, int m)
{
    if (n > m)
    {
        return findMedianSortedArrays(arr2, m, arr1, n);
    }

    int l = 0;
    int r = n - 1;
    int total = n + m;
    int half = total / 2;

    while (1)
    {
        int i = (l + r) / 2;         // pointer for arr1
        int j = half - (i + 1) - 1; // pointer for arr2

        int left1 = (i >= 0) ? arr1[i] : INT_MIN;
        int right1 = ((i + 1) < n) ? arr1[i + 1] : INT_MAX;

        int left2 = (j >= 0) ? arr2[j] : INT_MIN;
        int right2 = ((j + 1) < m) ? arr2[j + 1] : INT_MAX;

        // partition is correct
```

```c
        if (left1 <= right2 && left2 <= right1)
        {
            // odd
            if (total % 2 == 1)
            {
                return (double)MIN(right1, right2);
            }
            else
            {
                return (MAX(left1, left2) + MIN(right1, right2)) / 2.0;
            }
        }
        else if (left1 > right2)
        {
            // reduce size of arr1
            r = i - 1;
        }
        else
        {
            l = i + 1;
        }
    }
}

int main()
{
    int bankA_transactions[] = {1200, 1500, 2000, 2500, 3000};
    int n = sizeof(bankA_transactions) / sizeof(bankA_transactions[0]);

    int bankB_transactions[] = {1100, 1800, 2100, 2400, 2900, 3200};
    int m = sizeof(bankB_transactions) / sizeof(bankB_transactions[0]);

    printf("Bank A Data: ");
    for (int i = 0; i < n; i++)
        printf("%d ", bankA_transactions[i]);
    printf("\n");

    printf("Bank B Data: ");
    for (int i = 0; i < m; i++)
        printf("%d ", bankB_transactions[i]);
    printf("\n");

    double median = findMedianSortedArrays(bankA_transactions, n, bankB_transactions, m);

    printf("\nMedian Transaction Value: %.2f\n", median);

    return 0;
```

```
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment4> gcc .\Ques3.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment4> ./a.exe
Bank A Data: 1200 1500 2000 2500 3000
Bank B Data: 1100 1800 2100 2400 2900 3200

Median Transaction Value: 2100.00
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment4>
```

Figure 4: Output for Problem 3.

Analysis

- **Efficiency:** The algorithm uses binary search on the smaller array. By partitioning both arrays such that the left side contains the smaller half of all elements, we find the median in $O(\log(\min(n, m)))$ time.
- **Memory:** Space complexity is $O(1)$ (ignoring recursion stack if iterative), satisfying the constraint of not merging arrays.
- **Conclusion:** This is the optimal approach for merging streams or processing data from multiple sorted sources without incurring the linear cost of a merge operation.